

Button Mic Wire Polarity Detection

Project Report — Soham Bhattacharyya

1. Project Overview

This project extends the BluArmor AOI platform to a second inspection task: verifying wire polarity on a button mic PCB. The board has two soldered wires — red (positive) and gold (negative) in the original configuration — and the system checks whether red is connected to the right/positive terminal and gold to the left/negative terminal. The board's asymmetric shape rules out orientation confusion if flipped.

The task was deliberately built to reuse the existing connector-inspection pipeline with minimal changes: the same two-class scheme (defective/passing), the same verdict logic, and the same touchscreen application, extended to support switching between inspection tasks on one Raspberry Pi 5.

2. Project Workflow

- Define class scheme and labelling convention, reusing the connector project's defective/passing classes
- Collect and annotate ~250 sample images per class (defective, passing)
- Build and correct the Kaggle training pipeline (dataset export, split, augmentation, training, evaluation, packaging)
- Train a dedicated YOLO11n model from COCO-pretrained weights
- Restructure the Raspberry Pi deployment to support multiple inspection tasks on one device
- Calibrate camera focus for the mic board's close (macro-range) working distance
- Build continuous auto-detect inference, removing the need for a manual capture button
- Deploy to the production line and monitor real-world performance

3. Model & Dataset

Class	Definition
passing	Red wire on the right/positive terminal, gold wire on the left/negative terminal
defective	Wires swapped relative to the correct polarity

Key design decisions:

- One bounding box per image around the wire-terminal junction region, not the whole board
- `flipplr=0.0` during training — the camera/jig always presents the board in one fixed orientation, so a flipped training image would show red on a physically impossible side
- Trained from `yolo11n.pt` (COCO-pretrained) rather than fine-tuned from the connector checkpoint, since this is a different physical object with no shared visual foundation worth reusing
- A Roboflow export quirk (this dataset version had no valid/test folders — 100% train) was corrected in the pipeline by skipping any split that isn't found on disk, since the pipeline's own stratified re-split rebuilds train/val/test from the pooled images regardless

4. Model History

Version	Description	Status
aoi_mic_wiring_v1	Initial model, trained from COCO-pretrained YOLO11n on ~250 images/class, 70/20/10 split, fliplr=0.0	Deployed to production

5. Deployment Architecture

- Multi-task Raspberry Pi 5 setup — connector and mic-wiring inspection share one device, switchable via the touchscreen UI, each with its own model, log files, and camera settings
- Manual focus locked to a calibrated value for the mic-wiring task specifically, found via focus bracketing, since the task's working distance is in macro range and autofocus proved unreliable there
- Continuous auto-detect inference loop — polls the camera and automatically triggers a logged inspection when a board is placed, with debounce logic so a stationary board doesn't re-trigger repeatedly
- Every inspection now logs an image regardless of verdict — passing results save to a separate passed-images folder, non-passing results continue saving to the existing flagged-images folder
- A funnel-style physical placement guide, added in response to stakeholder feedback (see Section 7.2), mechanically constrains each board to the correct focal position

6. Model Performance

The deployed model is achieving approximately 96.47% accuracy in production. This is a strong result, though it's worth noting the original training set was relatively small (~250 images per class), and unlike the connector project's formal live-validation process (a fixed known-board test set with a pre-committed decision rule), the mic-wiring model has not yet gone through that same structured validation protocol. Given production accuracy is already being tracked, formalizing this into a repeatable validation set is a reasonable next step (see Section 9).

7. Rachit Sir Feedback & Response

7.1 QR Code Integration — Attempted, Reverted

Following a suggestion to scan each unit's QR code together with the inference step, this was attempted using the pyzbar library integrated into the same capture loop as the defect inference. In practice this introduced two problems: a meaningful rate of missed/failed QR reads, and a significant latency increase — per-unit processing time rose from roughly 3 seconds to sometimes over 7 seconds. This is consistent with broader community reports of pyzbar's reliability under real-world conditions (skew, lighting, small code size).

Decision: revert to scanning the QR code separately from the inference step, using a dedicated QR scanner rather than decoding it from the same camera feed used for defect detection. This decouples the two operations, avoiding the latency and reliability cost of running QR decode inside the same continuous inference loop.

7.2 Funnel Placement Guide — Implemented

A funnel-style mechanical guide was built to ensure each mic board is placed at a consistent position and focal distance under the camera, addressing missed detections caused by inconsistent placement during continuous auto-detect operation. This has been built and tested, with positive results.

8. Current Problem: Wire Color Generalization

The production line has begun using additional wire color combinations (green/blue) alongside the original red/gold pairing the model was trained on. Since the model learned to associate specific colors with specific terminal positions, it produces unreliable results on these new color combinations — effectively a color-based domain shift the model was never trained to handle, similar in nature to the earlier white-connector generalization issue on the connector line, but not yet addressed for this task.

Option	Description
A: Standardize wire colors in production	Restrict the line to the original red/gold pairing the model was trained on. No ML work required; requires an upstream process/procurement change.
B: Expand the dataset and retrain	Collect a meaningful volume of images for each additional color combination and fine-tune the model to generalize across all colors in use, following the same additive approach used for the connector's black/white generalization.

9. Recommendations & Next Steps

- Decide between wire-color standardization and a multi-color dataset expansion; if expanding, apply the same combined-dataset fine-tuning approach validated on the connector project rather than training on new colors alone
- Integrate a dedicated, separate QR scanner into the production station rather than decoding QR from the inference camera feed
- Track production accuracy over time, especially through any wire-color or dataset changes, to catch regressions early

10. Timeline Summary

Phase	Status
Class scheme & labelling design	Complete
Data collection (~250 images/class)	Complete
Training pipeline & model v1	Complete
Multi-task Raspberry Pi deployment	Complete
Macro focus calibration	Complete
Continuous auto-detect inference	Complete
Production rollout	Complete — ~96.47% accuracy reported
Funnel placement guide	Complete
QR scanning (in-pipeline, via pyzbar)	Attempted — reverted due to latency/reliability
Wire color generalization	Open — decision pending